

Mini-projet 2 — Structures de données + jeu de tests

Semaine 14 ■ Seul ou en binôme · Note sur 20 ■ TP de 2 h + finition maison

Le principe

Construire une application autour d'une **structure de données bien choisie**, avec la méthode de la semaine 13 : chaque fonction est **spécifiée, protégée et testée**. Rendu en début de semaine suivante (veille des vacances : les retards fondront comme neige...).

Sujet A — Le carnet de notes

Les données : un dictionnaire élève $\rightarrow$ liste de notes, {"Ada": [15, 12], "Blaise": [9]}.

Fonctions attendues : `ajouter_note(carnet, eleve, note)` (crée l'élève au besoin; précondition $0 \leq \text{note} \leq 20$ avec `assert`); `moyenne_eleve(carnet, eleve)`; `moyenne_classe(carnet)` (moyenne des moyennes); `meilleur(carnet)`; `en_difficulte(carnet, seuil)` (liste des élèves sous le seuil). Menu final : saisir, consulter, bilan.

★ **Bonus** : `bilan(carnet)` affiche un tableau aligné (f-strings {"...:<10"}) trié alphabétiquement.

Sujet B — Le morpion qui ne perd jamais (ou presque)

Reprendre le morpion du TP 12 et le terminer en jeu contre l'ordinateur.

Fonctions attendues : celles du TP 12 **testées** (la grille étant un paramètre, `victoire` se teste très bien!); un adversaire `coup_ordinateur(g)` en trois réflexes : jouer le coup gagnant s'il existe, sinon bloquer le coup gagnant adverse, sinon case libre (le centre d'abord). Partie complète avec choix du symbole.

★ **Bonus** : score sur plusieurs manches; grille de taille n paramétrable.

Sujet C — Le vérificateur de mots de passe

Les données : des chaînes, des listes, et un rapport sous forme de dictionnaire.

Fonctions attendues : `longueur_ok(mdp)` (≥ 10); `contient_chiffre(mdp)`, `contient_majuscule(mdp)`, `contient_special(mdp)` (parcours de chaîne, `c in "..."`); `dans_liste_noire(mdp, interdits)` (liste des 20 mots de passe les plus courants, à chercher en ligne); `score(mdp)` sur 100 combinant le tout, avec un **rapport** {"longueur": True, ...}; verdict final en f-strings.

★ **Bonus** : `suggestion(mdp)` propose une amélioration (ajouter la longueur manquante, substituer des caractères...).

Exigences communes — la nouveauté de ce projet

1. Toute fonction porte docstring + préconditions (`assert`).
2. Un fichier `tests.py` séparé : une fonction `test_xxx()` par fonction du projet, **cas limites inclus**; tout passe au vert.
3. Programme principal court, travail dans les fonctions, en-tête complet.
4. Le fichier `tests.md` de la période 1 devient inutile : `tests.py` EST votre preuve de tests.

Barème (20 points)

Critère	Points
Le programme fonctionne (scénario normal sans plantage)	5
Structure de données adaptée, fonctions pertinentes	4
Jeu de tests : couverture et cas limites	4
Docstrings, préconditions, asserts	3
Attendus spécifiques du sujet	2
Lisibilité, en-tête	2
Bonus ★	+2 max